Labo 4: Coalesced memory access and device (GPU) memory types
Geavanceerde computerarchitecturen

Jan Martens & Leon Cherchye

November 20, 2024



Session Date: 15 november, 2024

Class: Ma ELICT optie software-engineering

1 Introduction

In dit rapport analyseren we de prestaties van GPU-geheugentypen (globaal, gedeeld en constant) en de impact van gecoördineerde geheugentoegang op de uitvoeringstijd. Ook onderzoeken we de schaal-effecten van blokgrootte en invoergrootte in matrixvermenigvuldigings-kernels.

2 Coalesced memory access

2.1 Specificatie van de GPU Nvidia MX450

De GPU die ik gebruikte, heeft 896 cores en 14 streaming multiprocessors. Dit geeft dus een warp-grootte van 64 cores.

2.2 Verschil in performance tussen een coalesced en uncoalesced kernel

Het verschil in performance tussen een coalesced en een uncoalesced kernel is duidelijk te zien in de uitvoeringstijd. Een coalesced kernel heeft significant minder tijd nodig om volledig uit te voeren. Dit is volledig te wijten aan het feit dat een coalesced kernel maar één geheugentoegang nodig heeft om alle gegevens voor de threads in die warp op te halen. Hieruit kunnen we afleiden dat een significant deel van de uitvoeringstijd te wijten is aan geheugentoegang.

2.3 Invloed van input-grootte en grid-grootte op performance (verschil tussen coalesced en uncoalesced)

Wanneer we de input-grootte van de array aanpassen, en er dus meer geheugentoegangen nodig zijn, zien we dat het verschil in uitvoeringstijd groter wordt. Dit is logisch, want hoe meer geheugentoegangen geparallelliseerd kunnen worden, hoe meer voordeel er gehaald kan worden uit een coalesced kernel.
Verder zien we dat het verschil in uitvoeringstijd het grootste is als de gridsize zo gekozen is dat het aantal threads een geheel in aantal warps gescheduled kan worden. Uncoalesced kernels halen veel minder voordeel uit het optimaliseren van het aantal threads per block. Dit is te zien aan de quasi-constante uitvoeringstijd, onafhankelijk van het aantal threads per block. Dit is te verklaren doordat de sequentiele memory-access per warp is, en niet per block. Als er door een slechte keuze in block-size, dus bijvoorbeeld één thread te veel is waardoor alle andere cores in de warp moeten wachten op die ene thread, wordt er tijd verspilt door het wachten, maar is er daarbovenop ook een extra memory-access nodig voor deze ene thread. Deze extra memory-access neemt het gros van de tijd in beslag. Dat verklaard waarom de uitvoeringstijd veel constanter is bij uncoalesced memory-accesses: Elke thread moet toch sowieso appart een memory-access doen, waardoor het nadeel van de wachtende cores veel minder zichtbaar is.

3 Global, shared, and constant memory

3.1 Tijd voor Geheugenmigratie (CPU naar GPU Global & GPU Global naar Shared Memory)

Voor deze analyse hebben we een specifiek programma geschreven, part2_memoryMigration.cu, dat alleen gericht is op het schalen en timen van geheugenmigratie. Hierbij vergroten we de te

migreren datagrootte stapsgewijs van 0 MB tot 2 GB met een stapgrootte van 10 MB.
De resultaten zijn gevisualiseerd in Figuur 3. Zoals te zien is, vertonen beide geheugenmigraties een lineair verloop, maar met een sterk verschillende helling. Voor CPU → GPU Global Memory is de helling 0.161ms/MB, terwijl voor GPU Global → Shared Memory deze slechts 0.009ms/MB bedraagt.
Hieruit kunnen we concluderen dat geheugenmigratie in de tweede situatie (GPU Global → Shared Memory) ongeveer 18 keer sneller verloopt dan in de eerste situatie.

3.2 Schaling met blokgrootte matrixvermenigvuldiging

We vergroten de blokgrootte van 16x16 naar 32x32 tot 64x64. (Zie figuren 4, 5 en 6)

- **Global memory**: De uitvoeringstijd daalt naarmate de blokgrootte vergroot. Bij een invoergrootte van 256x256 zien we een afwijking, hier is er een sterke daling van 16x16 naar 32x32. *Mogelijke verklaring:* Bij kleinere invoermatrices (256x256) is de totale hoeveelheid data kleiner, waardoor de relatieve overhead voor global memory-toegang minder uitgesproken wordt. Dit effect verdwijnt bij grotere invoermatrices.

- **Shared memory**: De uitvoeringstijd daalt ook consistent naarmate de blokgrootte vergroot. Dit komt doordat shared memory efficiënter is in datahergebruik binnen een threadblok en grotere blokken zorgen voor minder frequent geheugenoverhead.

- **Constant memory**: De tijd stijgt sterk bij 32x32 en daalt terug sterk bij 64x64. *Mogelijke verklaring:* Dit kan liggen aan de werking van de constant cache, waar toegangspatronen bij 32x32 conflicten of inefficiënties veroorzaken. Bij 64x64 wordt mogelijk een optimalisatie toegepast of de grootte van blokken sluit beter aan bij de hardwarecapaciteiten. Een andere mogelijkheid is dat er een fout zit in de code die we niet gevonden hebbe. Deze opties zijn slechts hypotheses.

We kunnen concluderen dat grotere blokgroottes doorgaans leiden tot betere prestaties voor global en shared memory. Voor constant memory zijn de resultaten minder voorspelbaar door mogelijke cache-gerelateerde effecten.

3.3 Schaling met invoergrootte matrixvermenigvuldiging

We vergroten de grootte van de matrices van 256x256 naar 512x512 tot 1024x1024. (Zie figuren 4, 5 en 6)

- De trends in de grafieken blijven grotendeels consistent over de verschillende blokgroottes en invoergroottes. Dit wijst erop dat de prestaties van global, shared en constant memory grotendeels schaalbaar blijven.

- Een uitzondering zien we bij de uitvoertijden van global memory voor een invoergrootte van 256x256. Hier daalt de tijd sterker naarmate de blokgrootte toeneemt, vergeleken met grotere invoergroottes. Hiervoor is er al een mogelijke verklaring gegeven in de vorige subsectie.

We kunnen concluderen dat de implementaties over het algemeen goed schalen met grotere invoergroottes, zeker voor shared en constant memory. Global memory toont enige afwijkingen bij kleine invoergroottes, maar schaalt eveneens goed bij grotere matrices.

3.4 Bemangrijke instellingen matrixvermenigvuldiging

De constanten TILE_SIZE (= 64x64 elementen) en MAX_CONSTANT_SIZE (= 1024 elementen) zijn essentieel:

- TILE_SIZE bepaalt de grootte van blokken die gedeeld geheugen gebruiken, wat invloed heeft op de parallellisatie.
- MAX_CONSTANT_SIZE beperkt het aantal elementen dat in constant memory kan worden opgeslagen. Dit voorkomt overschrijding van de hardwarelimieten.

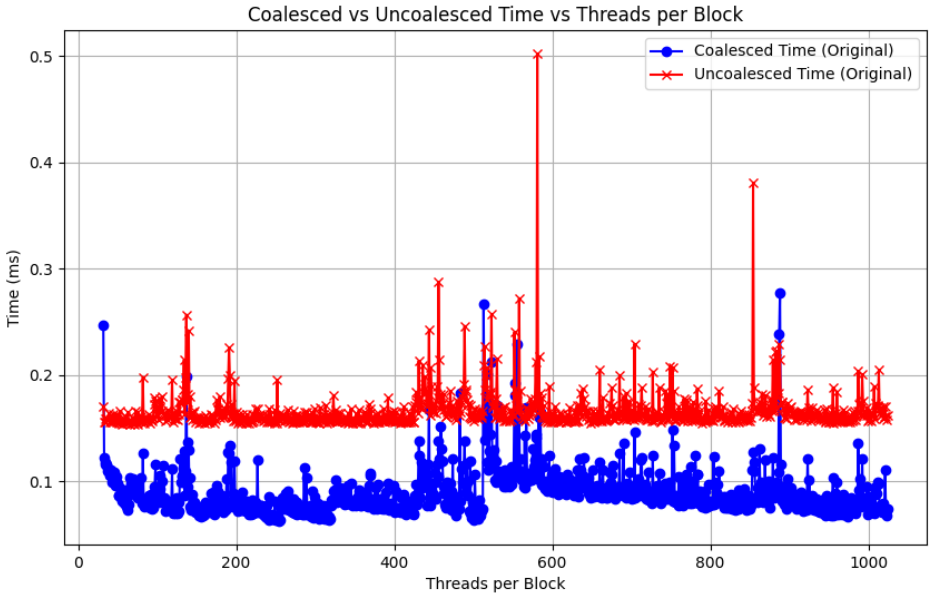


Figure 1: Kernel execution time vs threads per block (small input)

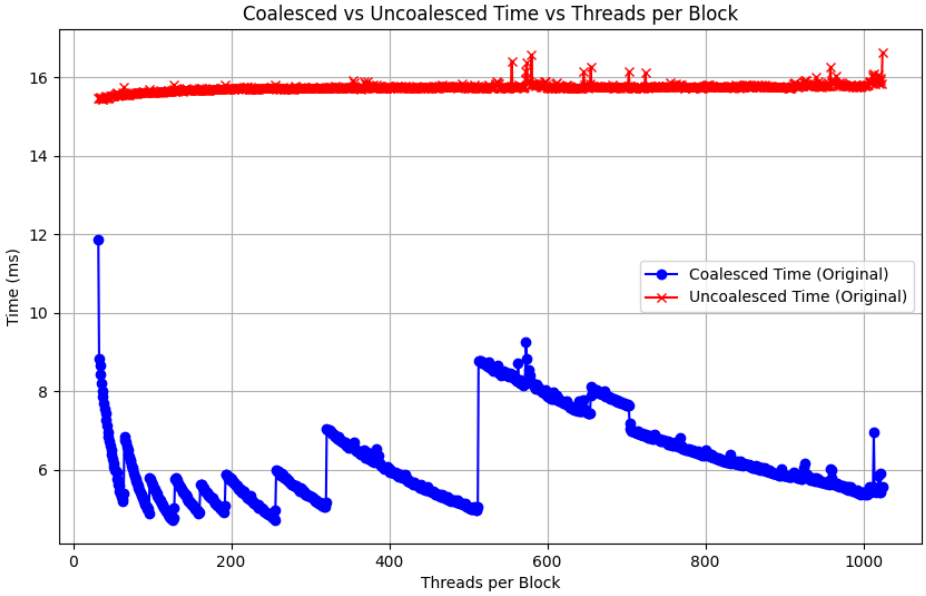


Figure 2: Kernel execution time vs threads per block (big input)

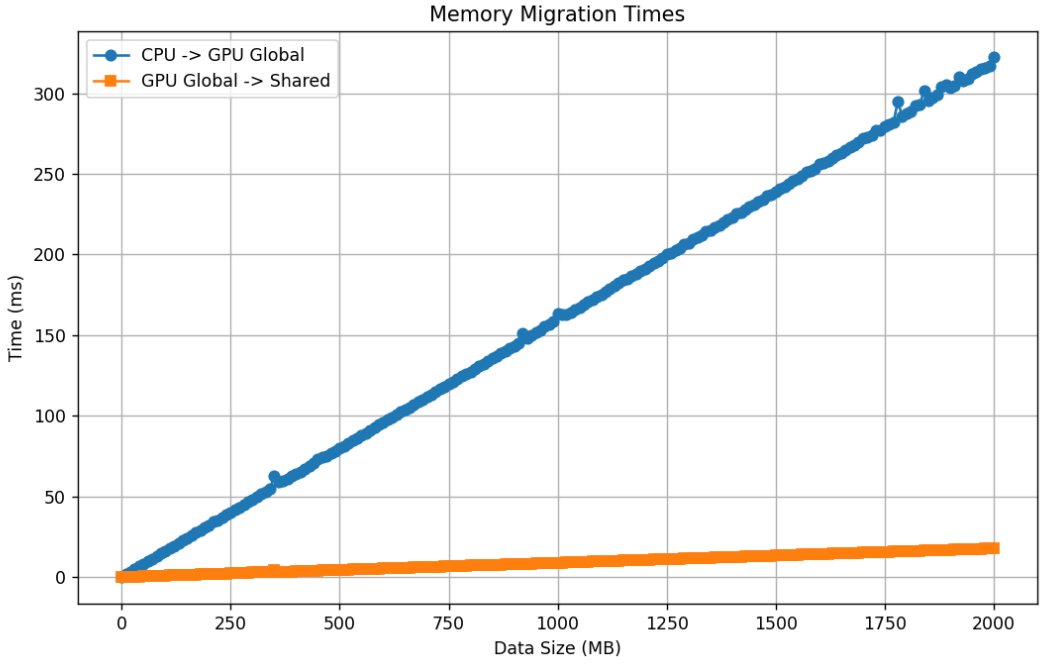


Figure 3: Memory migration time vs data size

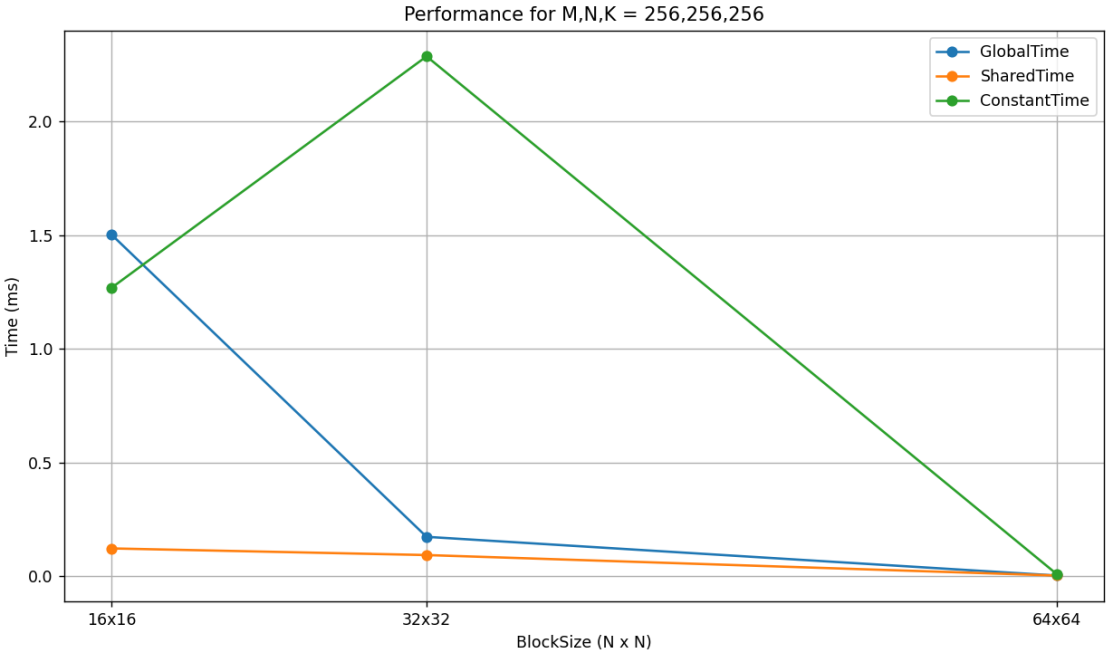


Figure 4: Kernel execution time vs input matrix 256x256

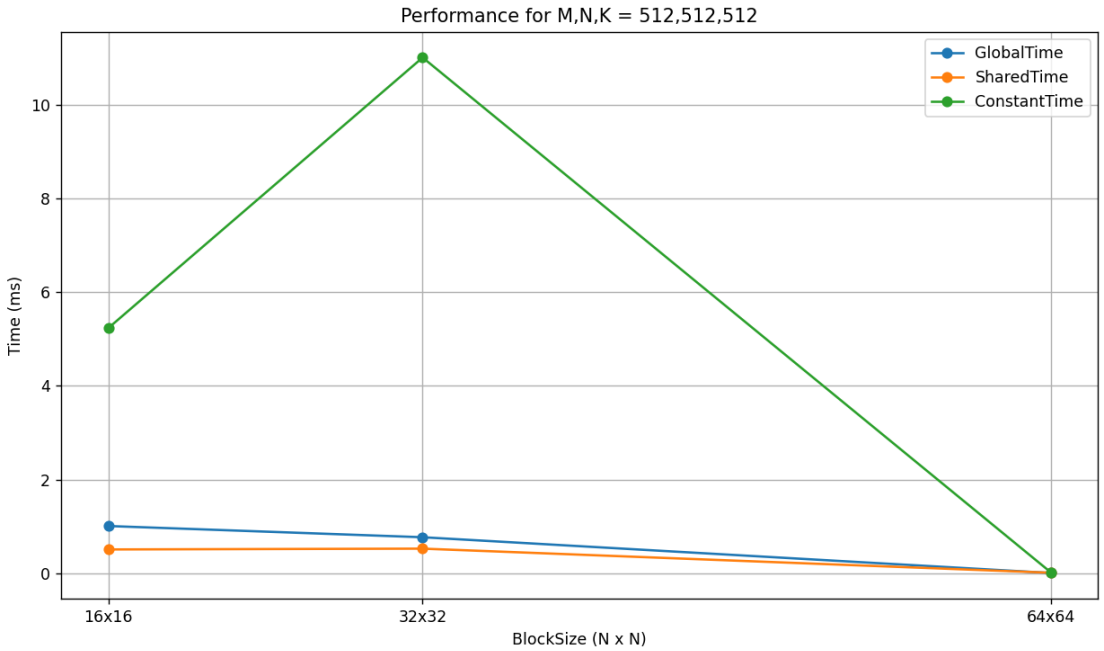


Figure 5: Kernel execution time vs input matrix 512x512

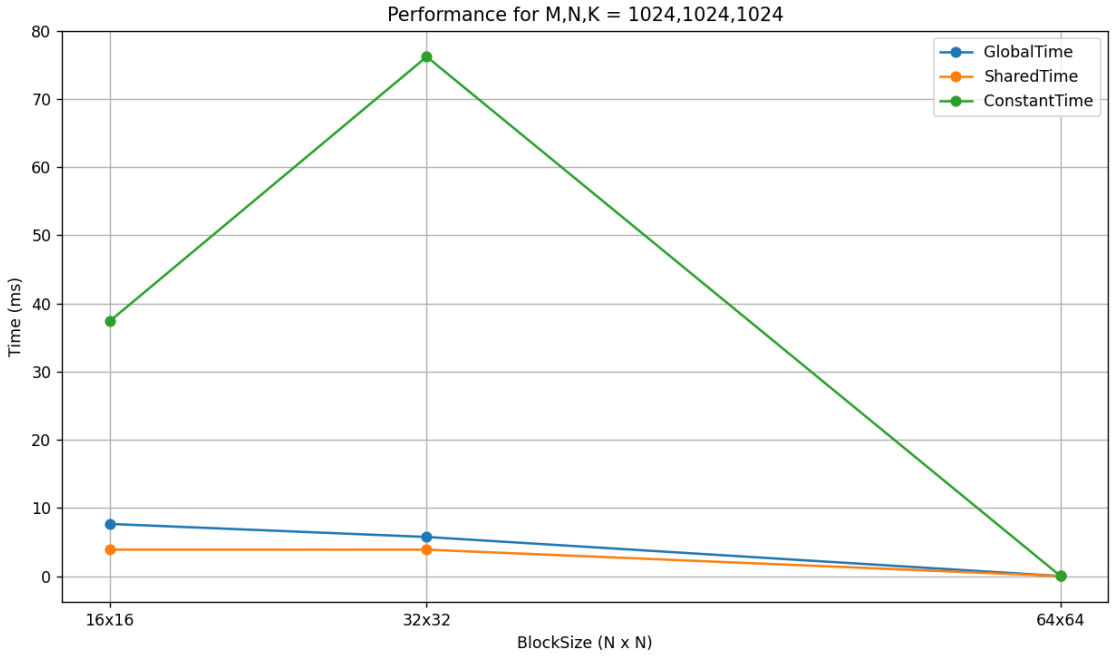


Figure 6: Kernel execution time vs input matrix 1024x1024